



**Universidad
Gerardo Barrios**



Comisión de Acreditación de la
Calidad de la Educación Superior
UNIVERSIDAD GERARDO BARRIOS (UGB)
ACREDITADA
2024-2029

**Universidad Gerardo Barrios
Facultad de Ciencia y Tecnología**

**Informe Final de Proyecto
Trabajo de Graduación
Modalidad Preespecialización**

en la Especialidad de:
Desarrollo de Aplicaciones con Inteligencia Artificial

**“NovaTareas Pro: Asistente Personal Inteligente de Gestión
de Tareas para Estudiantes Universitarios”**

Presentado por:

Saúl Oswaldo López Hernández
Moises Antonio Martínez
Enson Onan Carranza Rodríguez

San Miguel, agosto de 2026

ÍNDICE

ÍNDICE	2
RESUMEN DEL PROYECTO	3
MARCO CONCEPTUAL.....	6
1. Gestión del tiempo y carga académica	6
2. Inteligencia artificial generativa y modelos de lenguaje	7
3. Ingeniería de instrucciones	8
4. Generación aumentada por recuperación y representaciones vectoriales	9
5. Arquitectura de software: monolito modular y microservicios	10
6. Persistencia relacional y migraciones versionadas.....	11
7. Pruebas automatizadas	12
8. Integración y entrega continuas.....	13
9. Contenedores y reproducibilidad	14
10. Observabilidad.....	15
11. Seguridad en aplicaciones web	16
12. Versionado, publicación y vuelta atrás	17
13. Síntesis del marco conceptual	18
ANÁLISIS DEL PROBLEMA	20
1. Contexto	20
2. Evidencia recogida	20
3. Definición del problema	21
4. Limitaciones de las alternativas existentes	21
5. Formulación.....	22
PROPUESTA DE SOLUCIÓN.....	23

1. Descripción general	23
2. Justificación técnica de las decisiones principales	23
2.1 Arquitectura de monolito modular	23
2.2 Cascada de proveedores de inteligencia artificial	24
2.3 PostgreSQL como motor único	24
2.4 Verificación de correo en lugar de preguntas de seguridad	25
3. Modelo de datos	25
4. Modelo de permisos del trabajo compartido	26
METODOLOGÍA DE DESARROLLO	27
1. Enfoque general	27
2. Control de versiones y trazabilidad	27
3. Desarrollo guiado por pruebas.....	28
4. Estrategia de pruebas.....	29
5. Integración continua	29
6. Instrumentación y medición	31
7. Despliegue.....	31
RESULTADOS ALCANZADOS	33
1. Producto publicado y accesible	33
2. Calidad verificada	34
3. Rendimiento medido	35
4. Identificación del cuello de botella	35
5. Mejora aplicada y verificada	36
6. Plan de escalabilidad derivado de la medición	36
7. Evolución respecto al estado inicial	37
8. Limitaciones del resultado alcanzado	38

CONCLUSIONES GENERALES.....	39
1. Aporte a la comunidad.....	39
2. Resultados concretos obtenidos.....	39
3. Conocimientos adquiridos.....	40
4. Trabajo futuro	40
REFERENCIAS ESTUDIADAS	41
Bibliografía.....	41
Referencias web	41
Fuentes propias	42

RESUMEN DEL PROYECTO

NovaTareas Pro es un asistente personal inteligente de gestión de tareas dirigido a estudiantes universitarios, desarrollado como proyecto de graduación en la modalidad de preespecialización en Desarrollo de Aplicaciones con Inteligencia Artificial. El sistema se encuentra publicado y operando en <https://novatareas.polarzero.dev>, con dominio y certificado propios, y su código fuente está disponible en el repositorio <https://github.com/Saul1hdz/NovaTareas>.

El proyecto nace de un problema documentado con evidencia propia. Una encuesta aplicada en febrero de 2026 a estudiantes de la Universidad Gerardo Barrios mostró que el ochenta por ciento considera difícil o muy difícil organizar sus tareas, que el ochenta por ciento olvida al menos una entrega por ciclo académico y que entre el ochenta y el noventa por ciento percibe que esa desorganización afecta sus calificaciones. Las herramientas comerciales disponibles —Todoist, Notion o Trello, entre otras— cobran por sus funciones avanzadas, están pensadas en inglés y, sobre todo, no aprenden del historial de trabajo de quien las utiliza: ofrecen un lugar donde anotar, pero no un criterio sobre qué hacer primero ni cómo empezar.

La solución construida aborda esa carencia mediante tres vías de acceso que comparten una misma base de datos y una misma lógica de negocio: un panel web con renderizado en servidor, un bot conversacional de Telegram y una interfaz de programación protegida que expone la capacidad inteligente como servicio para clientes autorizados. El estudiante gestiona sus tareas con prioridad, etiquetas, fecha límite, subtareas y comentarios de avance; puede compartir una tarea privada mediante un enlace de invitación con tres niveles de acceso diferenciados; y recibe recordatorios cuya frecuencia depende de la prioridad de cada pendiente.

La inteligencia artificial cumple una función acotada y verificable: indicar por dónde conviene empezar una tarea concreta. Para garantizar que el servicio nunca deje al usuario sin respuesta, la generación de recomendaciones se organiza en una cascada de cuatro escalones. El primero consulta un modelo de lenguaje en la nube;

si este no responde o se agota la cuota contratada, interviene un modelo local; si tampoco está disponible, el sistema recupera información de las tareas que esa misma persona ya completó anteriormente mediante búsqueda por similitud semántica; y como último recurso aplica reglas heurísticas deterministas que no requieren conexión. Cada recomendación registra de qué escalón provino, lo que permite atribuir cualquier variación de calidad al nivel concreto que la produjo.

El desarrollo se sostuvo sobre prácticas de ingeniería verificables. El sistema cuenta con una suite de pruebas automatizadas distribuida en treinta y tres archivos, ejecutada contra un motor PostgreSQL real y no contra un sustituto simplificado; diez de ellas son deliberadamente adversariales y comprueban que ciertos comportamientos indebidos permanezcan bloqueados. Un proceso de integración continua se dispara con cada cambio del código: revisa tipos y sintaxis, prepara una base de datos limpia, ejecuta la totalidad de las pruebas con reporte de cobertura, compila el proyecto, construye la imagen de despliegue y la arranca para comprobar que responde antes de darla por válida.

La aplicación fue instrumentada para emitir un registro estructurado por cada solicitud atendida, con identificador de correlación, ruta normalizada, estado, duración y versión del código. Esa instrumentación permitió establecer una línea base de rendimiento sobre el flujo más utilizado —el listado de tareas del panel— con una mediana de dieciséis coma diecinueve milisegundos, un percentil noventa y cinco de veintitrés coma setenta y cuatro milisegundos y una tasa de error del cero por ciento. La medición reveló además un hallazgo que no resultaba evidente por inspección: cerca de tres cuartas partes del tiempo percibido por el cliente se consumían fuera del programa, en la capa de red del entorno de desarrollo. Sin esa evidencia, el esfuerzo de optimización se habría dirigido al lugar equivocado.

El proyecto declara con la misma claridad lo que no alcanzó. La publicación de nuevas versiones conserva pasos manuales; el proceso de observación con estudiantes para medir el efecto sobre hábitos y calificaciones no se ha completado; la integración con calendarios externos está construida y probada pero no conectada

a la interfaz principal; y la pantalla del panel quedó concentrada en un archivo de dimensiones excesivas que conviene dividir. Estas limitaciones se documentan en el cuerpo del informe, en el registro de cambios del repositorio y en el manifiesto de la versión publicada.

Los aportes principales del trabajo son tres. Primero, una aplicación real y accesible públicamente que resuelve una necesidad verificada en su comunidad inmediata, sin costo para el estudiante y en su idioma. Segundo, un diseño de degradación progresiva ante fallos de servicios externos, que convierte una dependencia frágil — el acceso a un modelo comercial— en un riesgo administrado. Y tercero, un conjunto de prácticas de ingeniería documentadas y reproducibles: pruebas contra el motor real, integración continua que verifica la imagen desplegable, medición del rendimiento antes de optimizar, y un procedimiento de vuelta atrás con sus límites explícitos.

MARCO CONCEPTUAL

Este apartado reúne los fundamentos teóricos sobre los que se levanta el proyecto. No pretende ser una revisión exhaustiva de cada disciplina, sino explicar los conceptos que se aplicaron efectivamente en la construcción de NovaTareas Pro y las razones por las cuales se eligieron unos enfoques y se descartaron otros.

1. Gestión del tiempo y carga académica

La gestión del tiempo en el contexto universitario ha sido estudiada como un factor asociado al rendimiento académico y al bienestar del estudiante. La sobrecarga percibida no proviene únicamente del volumen de trabajo, sino de la dificultad para establecer prioridades entre tareas que compiten por los mismos recursos cognitivos y temporales. Un estudiante que cursa varias asignaturas simultáneamente enfrenta un conjunto heterogéneo de compromisos —exámenes, proyectos grupales, lecturas, entregas parciales— cuya urgencia relativa cambia con el tiempo y rara vez resulta evidente a primera vista.

La literatura sobre productividad personal ha popularizado métodos de organización que comparten un principio: externalizar los compromisos a un sistema de confianza libera capacidad mental para el trabajo efectivo. El método conocido como *Getting Things Done*, propuesto por David Allen, articula ese principio en un flujo de captura, clarificación, organización, revisión y ejecución. Otras aproximaciones, como la matriz de Eisenhower, ofrecen criterios de priorización basados en la combinación de urgencia e importancia.

Estos métodos, sin embargo, presuponen una disciplina de revisión periódica que muchos estudiantes no sostienen. El sistema de confianza se degrada cuando la captura es incompleta o cuando la revisión se posterga. De ahí el interés de un asistente que no solo almacene, sino que también sugiera, recuerde y priorice: un sistema que compense parcialmente la falta de disciplina de revisión en lugar de asumirla como requisito.

En el contexto salvadoreño se añade una condición particular. Una proporción considerable de estudiantes universitarios combina el estudio con actividad laboral o con responsabilidades familiares, lo que reduce el tiempo disponible para la planificación y aumenta el costo de un olvido. Además, el acceso a herramientas de pago está limitado por consideraciones económicas y por la disponibilidad de medios de pago internacionales. Estas restricciones orientaron dos decisiones del proyecto: que la herramienta fuera gratuita para el usuario final y que estuviera disponible también por un canal de mensajería de uso cotidiano, reduciendo la barrera de adopción.

2. Inteligencia artificial generativa y modelos de lenguaje

Los modelos de lenguaje de gran escala son sistemas estadísticos entrenados para predecir la continuación más probable de una secuencia de texto. Su arquitectura dominante, el *transformer*, fue introducida por Vaswani y colaboradores en 2017 y sustituyó a las redes recurrentes en la mayoría de tareas de procesamiento de lenguaje natural. El mecanismo central de esta arquitectura, la atención, permite que cada elemento de la secuencia pondere su relación con todos los demás sin depender del orden secuencial de procesamiento, lo que habilitó el entrenamiento eficiente sobre corpus de gran tamaño.

Un modelo así entrenado no almacena hechos en el sentido de una base de datos ni razona mediante reglas explícitas. Genera texto plausible según las regularidades aprendidas. Esta caracterización tiene dos consecuencias prácticas que condicionaron el diseño del proyecto. La primera es que el modelo puede producir afirmaciones incorrectas con la misma fluidez que las correctas, fenómeno habitualmente denominado alucinación. La segunda es que su comportamiento depende críticamente de la información que se le proporciona en la solicitud, puesto que carece de acceso al contexto particular del usuario salvo que este se le suministre.

Para el caso de uso de NovaTareas Pro, ambas consecuencias resultan manejables. La tarea encomendada al modelo —sugerir un primer paso concreto para abordar una tarea descrita por el usuario— tolera cierta variabilidad en la respuesta y no exige exactitud factual sobre el mundo. Una recomendación imperfecta sigue siendo útil como punto de partida, a diferencia de lo que ocurriría en un dominio médico o legal. Esta consideración fue determinante para acotar la función de la inteligencia artificial dentro del sistema en lugar de extenderla a tareas donde el error tendría un costo mayor.

El modelo empleado como proveedor principal es GLM-4.5-flash, accesible mediante la interfaz de programación de z.ai. La elección atendió a tres criterios: disponibilidad de una capa gratuita o de bajo costo compatible con un proyecto académico, latencia adecuada para una interacción sincrónica, y calidad suficiente en idioma español. Como se detalla más adelante, la arquitectura no queda atada a ese proveedor: la interfaz de invocación está aislada, de modo que sustituirlo requiere modificar un único módulo.

3. Ingeniería de instrucciones

La ingeniería de instrucciones, conocida como *prompt engineering*, estudia cómo formular la entrada de un modelo generativo para obtener respuestas útiles y consistentes. En ausencia de reentrenamiento, la instrucción es el principal mecanismo de control sobre el comportamiento del modelo.

Las técnicas relevantes para este proyecto fueron tres. La primera es la asignación explícita de rol y de formato de salida, que reduce la dispersión de las respuestas y facilita su presentación uniforme en la interfaz. La segunda es la inyección de contexto, que consiste en incorporar a la instrucción información específica del caso —en este proyecto, los datos normalizados de la tarea, el tipo de usuario y, cuando existe, la evidencia recuperada del historial—. La tercera es la delimitación de la extensión y del alcance de la respuesta, necesaria tanto para controlar el consumo

de la cuota contratada como para que el resultado quepa en la interfaz sin desbordarla.

Un aspecto frecuentemente descuidado es el versionado de la instrucción. Una plantilla de instrucción que cambia sin dejar constancia invalida la comparación entre mediciones tomadas en momentos distintos: dos resultados separados por una modificación de la plantilla no son comparables, aunque lo parezcan. Por esa razón el proyecto identifica la plantilla vigente con una etiqueta de versión que se registra junto a cada evento del sistema, de modo que cualquier variación de calidad pueda atribuirse al cambio concreto que la originó.

4. Generación aumentada por recuperación y representaciones vectoriales

La generación aumentada por recuperación, conocida por sus siglas en inglés como RAG, fue formalizada por Lewis y colaboradores en 2020 como una arquitectura que combina un componente de recuperación de información con un modelo generativo. En lugar de confiar únicamente en el conocimiento incorporado durante el entrenamiento, el sistema busca fragmentos relevantes en una fuente externa y los incorpora a la instrucción, de modo que la generación quede fundamentada en información concreta y actualizable.

El componente de recuperación se apoya habitualmente en representaciones vectoriales, denominadas *embeddings*. Un modelo de representación transforma un fragmento de texto en un vector de dimensión fija, construido de manera que fragmentos con significado similar queden próximos en el espacio vectorial. La proximidad se mide típicamente mediante similitud del coseno, que compara la orientación de dos vectores con independencia de su magnitud. Esta propiedad permite recuperar contenido semánticamente relacionado aunque no comparta las mismas palabras, superando la limitación de la búsqueda por coincidencia literal.

En NovaTareas Pro esta técnica se aplica sobre el historial del propio usuario. Cuando una persona solicita una recomendación para una tarea nueva, el sistema

puede recuperar tareas que esa misma persona completó anteriormente y cuya descripción resulte semánticamente cercana, para ofrecerlas al modelo como evidencia de cómo suele trabajar. La diferencia con un asistente genérico es sustantiva: la sugerencia deja de basarse exclusivamente en patrones generales aprendidos durante el entrenamiento y se apoya en el comportamiento observado del usuario concreto.

Conviene precisar el estado real de esta capacidad en el proyecto. La infraestructura de almacenamiento vectorial está construida y las representaciones se generan y se persisten, pero no todas las rutas del sistema las consumen todavía. Se trata de una capacidad parcialmente integrada, y así se declara en el apartado de limitaciones.

5. Arquitectura de software: monolito modular y microservicios

La discusión entre arquitecturas monolíticas y distribuidas ha ocupado un lugar central en la ingeniería de software de la última década. Un monolito es una aplicación que se compila, se despliega y se escala como una unidad. Una arquitectura de microservicios descompone el sistema en servicios independientes que se comunican por red y se despliegan por separado.

La ventaja habitualmente atribuida a los microservicios —escalado independiente, despliegue autónomo, aislamiento de fallos— tiene un costo que suele subestimarse: latencia de red donde antes había llamadas a función, coordinación de despliegues, consistencia distribuida y una infraestructura de observabilidad considerablemente más compleja. Martin Fowler propuso al respecto el criterio conocido como *monolith first*: comenzar por un monolito bien modularizado y extraer servicios solo cuando exista evidencia concreta que lo justifique, puesto que resulta más sencillo dividir un sistema cuyas fronteras internas ya se comprenden que rehacer una descomposición prematura y equivocada.

El concepto de monolito modular recoge esa recomendación. La aplicación se despliega como una unidad, pero internamente mantiene fronteras estrictas entre capas con responsabilidades separadas: la capa de rutas traduce peticiones del

protocolo web en operaciones del dominio; la capa de dominio contiene las reglas de negocio y desconoce el protocolo por el que llegó la solicitud; la capa de persistencia ejecuta las consultas y no aplica reglas de negocio; y la capa de inteligencia artificial produce recomendaciones sin conocer la base de datos ni la sesión del usuario.

Esta última frontera resultó especialmente productiva en el proyecto. Precisamente porque el motor de recomendaciones no depende del modelo de usuarios ni del almacenamiento, la misma capacidad pudo exponerse como interfaz de programación para clientes externos sin arrastrar consigo el resto del sistema. Una frontera bien trazada dentro de un monolito es, además, el trabajo previo que abarata una eventual extracción futura.

6. Persistencia relacional y migraciones versionadas

El modelo relacional, formulado por Edgar Codd en 1970, organiza los datos en relaciones con restricciones declarativas de integridad. Frente a alternativas no relacionales, ofrece dos garantías que resultaron decisivas para este proyecto: la integridad referencial, que impide estados inconsistentes como un comentario asociado a una tarea inexistente, y las transacciones con propiedades de atomicidad, consistencia, aislamiento y durabilidad, que permiten que una operación compuesta se aplique íntegramente o no se aplique en absoluto.

PostgreSQL fue el motor seleccionado. Se trata de un sistema de gestión de bases de datos relacional de código abierto, con soporte para tipos de datos avanzados, control de concurrencia multiversión y extensiones que habilitan, entre otras capacidades, el almacenamiento y la búsqueda de vectores. Esta última característica permitió mantener las representaciones vectoriales en el mismo motor que los datos transaccionales, evitando la introducción de un componente adicional.

Un aspecto operativo que suele quedar fuera de los tratamientos introductorios es la evolución del esquema. Una base de datos en producción no puede recrearse desde cero cada vez que cambia la estructura: los datos existentes deben conservarse. Las migraciones versionadas resuelven este problema al expresar cada cambio del

esquema como un archivo ordenado y aplicable una sola vez, cuya ejecución queda registrada. Esta disciplina proporciona reproducibilidad —cualquier entorno alcanza el mismo esquema aplicando la misma secuencia— pero impone también una restricción importante que se discute más adelante: revertir el código de una aplicación no revierte automáticamente los cambios estructurales que ese código introdujo.

7. Pruebas automatizadas

Las pruebas automatizadas son programas que verifican el comportamiento de otros programas. Su valor no reside únicamente en detectar defectos, sino en documentar de forma ejecutable qué se espera del sistema: una prueba que falla señala con precisión qué comportamiento dejó de cumplirse.

La clasificación habitual distingue niveles según el alcance de lo que se ejercita. Las pruebas unitarias verifican una unidad de código aislada; las de integración comprueban la interacción entre componentes, incluida la base de datos; y las de extremo a extremo recorren el sistema completo desde la interfaz de usuario. La conocida pirámide de pruebas sugiere concentrar el mayor número en el nivel unitario, por su velocidad y estabilidad, y reservar los niveles superiores para los flujos críticos.

Una decisión metodológica relevante en este proyecto fue ejecutar las pruebas contra el mismo motor de base de datos que utiliza el entorno de producción, en lugar de sustituirlo por una alternativa más ligera. La práctica de emplear un motor distinto en pruebas acelera la ejecución, pero introduce una diferencia entre lo que se verifica y lo que finalmente se ejecuta: comportamientos ligados al dialecto, a los tipos de datos o a la concurrencia pueden divergir. Ejecutar sobre el motor real elimina esa clase de discrepancia al precio de una infraestructura de pruebas algo más pesada.

Otra decisión fue sustituir los servicios externos por dobles controlados. Las pruebas no invocan al proveedor de inteligencia artificial ni a la plataforma de mensajería: la

primera se configura sin credenciales, de modo que el sistema recurre a su respaldo determinista, y la segunda no dispone de credencial de envío. Con ello la suite se vuelve reproducible, no consume cuota y no depende de la disponibilidad de terceros.

Merece mención aparte la categoría de pruebas adversariales. En lugar de comprobar que una funcionalidad opera correctamente, verifican que una operación indebida permanezca bloqueada: acceder al recurso de otro usuario, modificar contenido sin el permiso adecuado, reutilizar un identificador de un solo uso o eludir un límite de intentos. Esta clase de prueba protege controles de seguridad que, por su naturaleza, no producen manifestaciones visibles cuando funcionan y por tanto pueden desaparecer inadvertidamente durante una refactorización.

8. Integración y entrega continuas

La integración continua es la práctica de incorporar los cambios de código a una rama compartida con alta frecuencia, verificando cada incorporación mediante un proceso automatizado. La entrega continua extiende ese principio hasta dejar el software en un estado permanentemente desplegable. Ambas prácticas fueron sistematizadas por Jez Humble y David Farley en su obra sobre entrega continua, y persiguen un objetivo común: reducir el intervalo entre la escritura de un cambio y la confirmación de que ese cambio no rompió nada.

El valor de esta disciplina se aprecia mejor por contraste. Sin un proceso automatizado, la verificación depende de que cada persona recuerde ejecutar las comprobaciones antes de integrar, con la consecuencia previsible de que los defectos se descubran tarde y de forma acumulada. Con un proceso automatizado, la verificación ocurre siempre, en un entorno limpio y de manera idéntica para todos.

Un aspecto de diseño frecuentemente omitido es hasta dónde debe llegar la verificación. Un proceso que se limita a ejecutar las pruebas comprueba el código, pero no comprueba el artefacto que efectivamente se despliega. En este proyecto se optó por extender el proceso hasta construir la imagen de producción y arrancarla,

verificando mediante una petición a la sonda de estado que el artefacto responde. Con ello se cubre una clase de fallo que las pruebas por sí solas no detectan: aquel en que el código es correcto pero el empaquetado o la configuración del contenedor no lo son.

La gestión de credenciales dentro del proceso automatizado merece atención específica. Un flujo de integración que utilizara credenciales reales expondría secretos en los registros de ejecución y consumiría cuotas de servicios de pago en cada verificación. La alternativa adoptada consiste en declarar valores ficticios para todas las variables sensibles y en dejar sin configurar el acceso a los proveedores externos, de modo que el sistema recurra a sus mecanismos de respaldo. El resultado es un proceso determinista, sin costo por ejecución y sin superficie de exposición.

9. Contenedores y reproducibilidad

Un contenedor empaqueta una aplicación junto con sus dependencias y su configuración en una unidad que se ejecuta de manera consistente en distintos entornos. A diferencia de la virtualización completa, comparte el núcleo del sistema anfitrión, lo que reduce sustancialmente el consumo de recursos y el tiempo de arranque.

La reproducibilidad es la propiedad que justifica su adopción en este proyecto. La expresión coloquial «en mi máquina funciona» describe una clase de fallo originada en diferencias no documentadas entre entornos: versiones distintas del intérprete, bibliotecas del sistema ausentes o variables de configuración divergentes. Al declarar el entorno completo en un archivo versionado junto al código, esas diferencias desaparecen o, al menos, quedan explícitas y sujetas a revisión.

La construcción por etapas, conocida como *multi-stage build*, permite separar el entorno necesario para compilar del entorno necesario para ejecutar. La imagen final incluye únicamente los artefactos compilados y las dependencias de ejecución, lo

que reduce su tamaño y su superficie de ataque al no incorporar herramientas de compilación en el sistema publicado.

La orquestación mediante archivos de composición describe de forma declarativa el conjunto de servicios que constituyen el sistema, sus dependencias de arranque, sus volúmenes persistentes y sus comprobaciones de salud. Esta descripción cumple una doble función: opera como configuración ejecutable y como documentación fidedigna de la arquitectura desplegada, con la ventaja de que no puede desactualizarse sin que el despliegue deje de funcionar.

10. Observabilidad

La observabilidad designa la capacidad de inferir el estado interno de un sistema a partir de las señales que emite. Se distingue del monitoreo tradicional en que este último comprueba condiciones previstas de antemano, mientras que la observabilidad aspira a permitir la formulación de preguntas nuevas sobre comportamientos no anticipados.

La práctica contemporánea reconoce tres clases de señal. Las métricas son valores numéricos agregados a lo largo del tiempo. Los registros son eventos discretos con marca temporal. Las trazas siguen el recorrido de una solicitud a través de los componentes de un sistema distribuido. En un sistema desplegado como unidad, como es el caso de este proyecto, el registro estructurado por solicitud proporciona la mayor parte del valor sin la complejidad de una infraestructura de trazado distribuido.

El registro estructurado consiste en emitir cada evento como un objeto de campos identificados, en lugar de una línea de texto libre. La diferencia es operativa: un campo identificado se puede filtrar, agregar y comparar; una línea de texto libre requiere análisis sintáctico frágil y propenso a errores. La correlación se logra asignando a cada solicitud un identificador único que acompaña a todos los eventos derivados de ella y que, devuelto al cliente en una cabecera de respuesta, permite localizar el registro exacto de un incidente reportado por un usuario.

La medición del rendimiento exige precisión conceptual sobre los estadísticos empleados. El promedio de la latencia resulta engañoso porque una minoría de respuestas lentas lo desplaza sin reflejar la experiencia mayoritaria. Los percentiles ofrecen una descripción más fiel: la mediana, o percentil cincuenta, indica el comportamiento típico, mientras que el percentil noventa y cinco describe la cola de respuestas lentas, que es la que determina la percepción de lentitud. La obra sobre ingeniería de confiabilidad publicada por Google desarrolla extensamente este punto y propone además el concepto de indicadores y objetivos de nivel de servicio como base para decidir cuándo un sistema requiere intervención.

Una consideración de privacidad atraviesa todo el diseño de la instrumentación. Los registros de una aplicación con datos personales pueden convertirse fácilmente en una fuente de filtración si se emiten sin control: cuerpos de petición, cabeceras de autenticación, direcciones de correo o identificadores presentes en las rutas. La estrategia adoptada consiste en una lista blanca de campos publicables y en la normalización de las rutas, sustituyendo identificadores numéricos y credenciales de un solo uso por marcadores genéricos antes de escribir el evento.

11. Seguridad en aplicaciones web

La seguridad de aplicaciones web se estructura habitualmente en torno a categorías de riesgo consolidadas, entre las que la clasificación publicada por el proyecto abierto de seguridad en aplicaciones web —OWASP— constituye la referencia más difundida. De sus categorías, cuatro resultaron directamente aplicables a este proyecto.

El control de acceso deficiente ocupa el primer lugar de esa clasificación. Se manifiesta cuando un usuario autenticado accede a recursos que no le pertenecen, habitualmente por confiar en identificadores enviados por el cliente sin verificar la pertenencia en el servidor. La contramedida consiste en filtrar por propietario o por permiso explícito en cada consulta, y no en ocultar los enlaces en la interfaz.

Los fallos de identificación y autenticación abarcan el almacenamiento inadecuado de contraseñas, la ausencia de límites de intentos y la gestión defectuosa de sesiones. El almacenamiento debe emplear funciones de derivación diseñadas para ser costosas de calcular, como bcrypt, de modo que una eventual filtración de la base no permita recuperar las contraseñas por fuerza bruta en tiempo razonable. Los límites de intentos deben persistir fuera de la memoria del proceso para sobrevivir a reinicios y para conservar su validez si el sistema escalara a varias instancias.

La falsificación de petición en sitios cruzados explota que el navegador adjunta automáticamente las credenciales de sesión a las peticiones dirigidas a un dominio, con independencia de qué página las originó. La defensa combina el atributo de cookie que restringe su envío desde contextos externos con la verificación en el servidor del origen declarado. Una consideración operativa importante surge cuando la aplicación opera detrás de un proxy inverso que termina la conexión cifrada: el servidor percibe la petición como no cifrada y, si compara el origen contra su propia percepción, rechaza peticiones legítimas. La solución consiste en atender las cabeceras que el proxy añade para informar del esquema original.

El fallo criptográfico se refiere a la exposición de datos sensibles por ausencia o mal uso del cifrado. En este proyecto la aplicación custodia credenciales de acceso a servicios de terceros, que se almacenan cifradas con un algoritmo de cifrado autenticado, de modo que su manipulación resulte detectable además de indescifrable.

Por último, la gestión de secretos constituye un problema operativo con implicaciones de seguridad. Un secreto incorporado al control de versiones debe considerarse comprometido de forma permanente, puesto que el historial conserva su valor aunque se elimine en un cambio posterior. La práctica correcta consiste en mantener los secretos fuera del repositorio, versionar únicamente un archivo de ejemplo con marcadores, y rotar cualquier credencial expuesta en lugar de limitarse a borrarla.

12. Versionado, publicación y vuelta atrás

El versionado semántico establece una convención de tres números —mayor, menor y parche— cuyo incremento comunica la naturaleza del cambio: el primero señala una modificación incompatible con la versión anterior, el segundo la incorporación de funcionalidad compatible y el tercero la corrección de defectos sin cambio de comportamiento. La utilidad de la convención depende de que exista un contrato definido respecto del cual juzgar la compatibilidad; en este proyecto, ese contrato es la interfaz de programación pública.

Una etiqueta de control de versiones asocia un nombre estable a un estado exacto e inmutable del código. Su función en la operación es doble: permite reproducir con exactitud lo que se publicó y ofrece un destino concreto al que retornar. Esta segunda función carece de valor si la etiqueta no se crea, motivo por el cual un procedimiento de vuelta atrás documentado que apunte a etiquetas inexistentes constituye una garantía solo aparente.

La vuelta atrás presenta un límite que conviene enunciar con precisión, porque su desconocimiento produce incidentes graves: revertir el código no revierte el esquema de la base de datos. Si la versión que se abandona aplicó migraciones estructurales, el código anterior encontrará un esquema que no comprende. El procedimiento correcto exige restaurar primero la copia de seguridad previa a la migración y desplegar después la versión anterior, en ese orden.

El manifiesto de publicación complementa la etiqueta al declarar de forma explícita qué compone exactamente una versión: el código, el contrato de la interfaz, el modelo de inteligencia artificial invocado, la versión de la plantilla de instrucción, el estado del esquema de datos, la imagen de contenedor y el conjunto de pruebas que la respalda. Esta declaración adquiere relevancia particular en sistemas que dependen de modelos generativos, donde el comportamiento observable puede variar sin que el código cambie, por efecto de una modificación en el proveedor o en la plantilla de instrucción.

13. Síntesis del marco conceptual

Los doce apartados anteriores no constituyen bloques independientes, sino un encadenamiento con una lógica común. La gestión del tiempo define el problema; los modelos de lenguaje y la ingeniería de instrucciones aportan el mecanismo de asistencia; la recuperación aumentada lo particulariza al historial de cada persona; la arquitectura modular y la persistencia relacional sostienen esa capacidad con garantías de integridad; las pruebas y la integración continua permiten modificar el sistema sin degradarlo; los contenedores hacen reproducible su ejecución; la observabilidad permite comprender su comportamiento real; la seguridad protege los datos que custodia; y el versionado con vuelta atrás acota el daño cuando algo sale mal.

El hilo que atraviesa todos ellos es la preferencia por la evidencia sobre la suposición. Cada decisión relevante del proyecto —el motor de base de datos, la ejecución de pruebas contra el motor real, la mejora de rendimiento aplicada, la sustitución del mecanismo de recuperación de cuentas— se adoptó a partir de una comprobación concreta y no de una preferencia general. El apartado de metodología documenta cómo se llevó esa orientación a la práctica.

ANÁLISIS DEL PROBLEMA

1. Contexto

La población objetivo del proyecto son los estudiantes universitarios de la Universidad Gerardo Barrios, en San Miguel, El Salvador. Se trata de un grupo que cursa varias asignaturas de manera simultánea y que, en una proporción significativa, combina el estudio con actividad laboral o con responsabilidades familiares. Esta combinación reduce el tiempo disponible para la planificación y eleva el costo de cada olvido, puesto que la recuperación de una entrega perdida compete con compromisos ya establecidos.

2. Evidencia recogida

Para caracterizar el problema con datos propios y no únicamente con supuestos, el equipo aplicó una encuesta a estudiantes de la institución durante febrero de 2026. Los resultados se resumen a continuación.

Tabla 1. Hallazgos principales de la encuesta aplicada en febrero de 2026

Aspecto consultado	Resultado
Dificultad percibida para organizar tareas	80 % la califica como regular, difícil o muy difícil; solo 10 % la considera fácil
Olvido de entregas por ciclo académico	80 % olvida entre una y tres; 20 % olvida más de tres; 10 % nunca olvida
Impacto percibido en las calificaciones	Entre 80 % y 90 % afirma que la desorganización afecta su rendimiento
Nivel de estrés por acumulación de tareas	80 % reporta estrés alto o muy alto
Método de organización predominante	Notas en el teléfono (40 % a 60 %), cuadernos o ausencia de método estructurado
Problemas más mencionados	Olvido de tareas, desorganización y falta de tiempo (30 % a 40 % cada uno)

Estos resultados coinciden en dirección con lo reportado por estudios sobre estrés académico en el ámbito latinoamericano, que sitúan entre el cuarenta y ocho y el setenta por ciento la proporción de estudiantes con estrés académico de nivel moderado a severo asociado a sobrecarga.

3. Definición del problema

Del análisis de la evidencia se desprende que el problema no consiste en la ausencia de herramientas donde anotar tareas. La mayoría de los encuestados ya emplea algún método, habitualmente notas en el teléfono. El problema radica en tres carencias específicas de esos métodos.

La primera es la ausencia de criterio de priorización. Una lista de pendientes no indica cuál abordar primero, y esa decisión debe tomarse repetidamente en condiciones de fatiga y presión, que son precisamente las peores condiciones para decidir bien.

La segunda es la ausencia de recordatorio proactivo. Una nota permanece inerte hasta que alguien la consulta; si la revisión se posterga, el sistema de organización deja de cumplir su función justo cuando más se necesita.

La tercera es la ausencia de acompañamiento en el inicio. Buena parte de la postergación se produce ante tareas percibidas como grandes o mal definidas, frente a las cuales la dificultad no está en recordar que existen sino en saber por dónde comenzar.

4. Limitaciones de las alternativas existentes

Las herramientas comerciales de gestión de tareas resuelven parcialmente la primera carencia y, con configuración, la segunda. Sin embargo, presentan tres barreras para la población objetivo. Su modelo de negocio reserva las funciones de priorización avanzada y automatización a planes de pago, cuyo costo recurrente en moneda extranjera resulta difícil de sostener para un estudiante. Su interfaz y su

documentación están concebidas primordialmente en inglés, lo que añade fricción de adopción. Y ninguna de ellas incorpora el historial de trabajo de la persona en la generación de sugerencias: ofrecen las mismas recomendaciones a cualquier usuario ante entradas equivalentes.

5. Formulación

El problema puede formularse así: los estudiantes universitarios disponen de medios para registrar sus tareas, pero carecen de un sistema accesible, en su idioma y sin costo, que priorice sus pendientes, se los recuerde de manera proactiva y les sugiera cómo comenzar, apoyándose en su forma real de trabajar.

PROPUESTA DE SOLUCIÓN

1. Descripción general

NovaTareas Pro es una aplicación web con renderizado en servidor, acompañada de un bot conversacional y de una interfaz de programación protegida. Las tres vías comparten una única base de datos y una única capa de lógica de negocio, de modo que una tarea creada desde el teléfono aparece de inmediato en el panel y puede ser consultada por un programa autorizado.

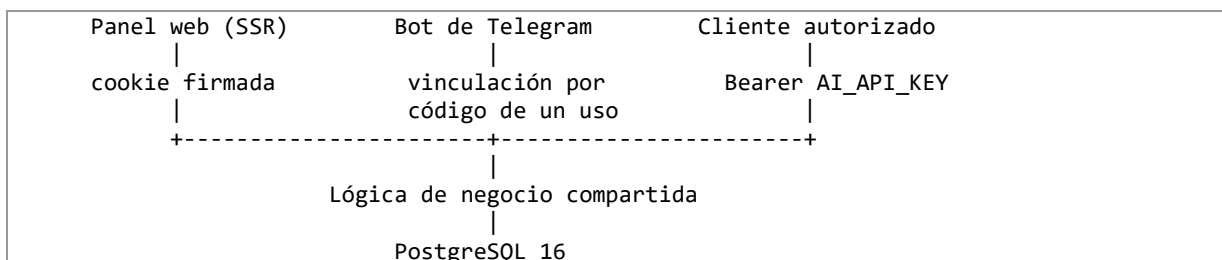


Figura 1. Vías de acceso al sistema y componentes que comparten

2. Justificación técnica de las decisiones principales

2.1 Arquitectura de monolito modular

Se optó por desplegar el sistema como una unidad con fronteras internas estrictas, en lugar de descomponerlo en servicios independientes. La justificación es doble. Por una parte, el equipo consta de tres personas y el volumen de uso previsto — decenas de usuarios concurrentes en los momentos de mayor actividad— cabe holgadamente en una sola instancia; introducir comunicación por red entre componentes añadiría latencia y modos de fallo sin resolver ningún problema existente. Por otra parte, las fronteras internas trazadas permiten extraer un servicio cuando exista evidencia que lo justifique, con un costo considerablemente menor que el de una descomposición prematura y mal ubicada.

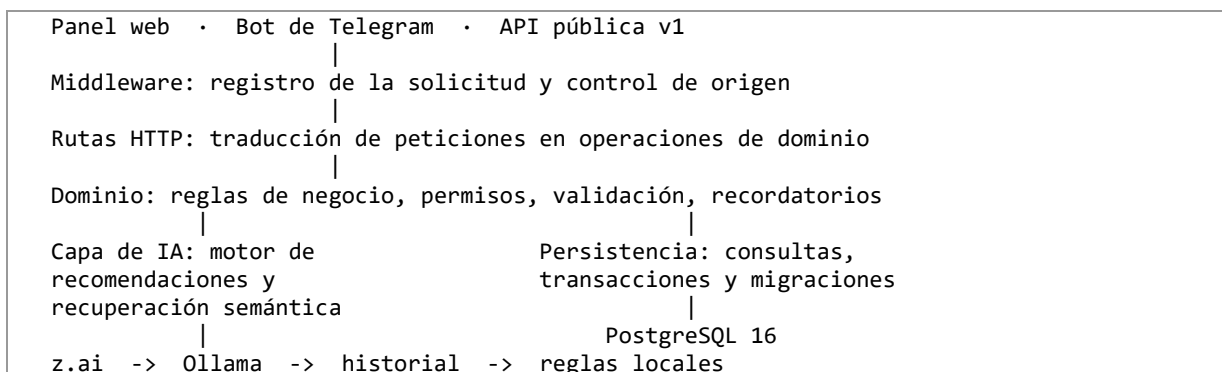


Figura 2. Arquitectura por capas del sistema desplegado

2.2 Cascada de proveedores de inteligencia artificial

La dependencia de un proveedor externo introduce un modo de fallo que escapa al control del equipo: una interrupción del servicio o el agotamiento de la cuota dejaría al sistema sin su función distintiva. En lugar de aceptar ese riesgo o de renunciar al modelo comercial, se diseñó una cascada de cuatro escalones con degradación progresiva.

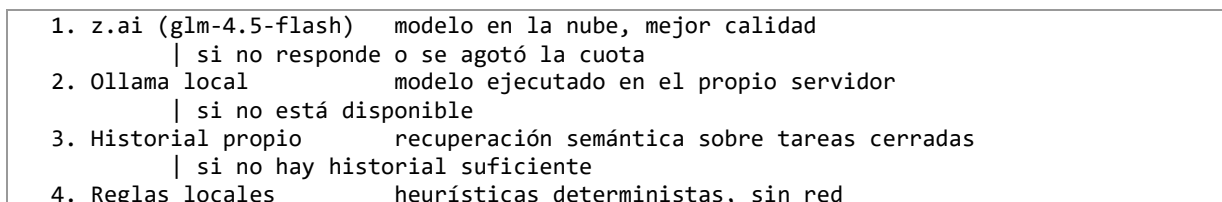


Figura 3. Cascada de generación de recomendaciones con degradación progresiva

Cada recomendación almacena el escalón que la produjo. Esta trazabilidad permite distinguir una caída de calidad atribuible al descenso de escalón de una atribuible a un cambio en la plantilla de instrucción, distinción imposible de establecer si el origen no se registrara.

2.3 PostgreSQL como motor único

El anteproyecto contemplaba emplear una plataforma gestionada de base de datos. Esa opción se descartó durante el desarrollo por dos razones. La primera es la dependencia operativa: una capa gratuita de un servicio de terceros puede modificar

sus límites o suspender proyectos inactivos, lo que introduciría un riesgo ajeno al control del equipo sobre el componente que custodia todos los datos. La segunda es la coherencia entre entornos: al gestionar el motor directamente, el entorno de pruebas, el de desarrollo y el de producción ejecutan la misma versión del mismo motor, eliminando una fuente conocida de discrepancias.

2.4 Verificación de correo en lugar de preguntas de seguridad

El mecanismo inicial de recuperación de cuentas se basaba en preguntas de seguridad. Se sustituyó por verificación mediante correo electrónico con enlaces de un solo uso, porque las respuestas a preguntas de seguridad son frecuentemente adivinables o averiguables, y porque el nuevo mecanismo permite además confirmar la titularidad de la dirección antes del primer acceso. La contrapartida, declarada como limitación, es la dependencia de un servidor de correo operativo.

3. Modelo de datos

El esquema comprende dieciocho tablas agrupadas según su propósito.

Tabla 2. Agrupación de las tablas del esquema según su propósito

Grupo	Contenido
Identidad	Usuarios, tokens de verificación de correo y tokens de recuperación
Tareas	Tareas, subtareas, historial de cambios, comentarios y categorías
Colaboración	Colaboradores de una tarea e invitaciones emitidas
Inteligencia	Recomendaciones generadas, valoraciones recibidas y representaciones vectoriales
Integraciones	Sesiones del bot y códigos de vinculación
Operación	Contadores de límites de intentos y registro de ejecuciones programadas

4. Modelo de permisos del trabajo compartido

Una tarea privada puede compartirse mediante un enlace de invitación con caducidad y de un solo uso. Cada persona invitada recibe uno de tres niveles de acceso.

Tabla 3. Niveles de acceso en el modo colaborativo

Nivel	Puede leer	Puede comentar	Puede editar	Puede eliminar
Lector	Sí	No	No	No
Comentarista	Sí	Sí	No	No
Editor	Sí	Sí	Sí	No
Propietario	Sí	Sí	Sí	Sí

La eliminación y la conversión de una tarea compartida en privada quedan reservadas al propietario. Esta última operación revoca el acceso de todos los colaboradores, comportamiento que se verifica mediante una prueba automatizada específica.

METODOLOGÍA DE DESARROLLO

1. Enfoque general

El desarrollo se organizó en bloques sucesivos, cada uno cerrado con un acta que registraba los resultados alcanzados, la evidencia que los respaldaba y los riesgos que quedaban abiertos. Este esquema, incremental y con puntos de control explícitos, permitió mantener el sistema en estado funcional durante todo el proceso y evitar la acumulación de trabajo sin verificar.

Tabla 4. Bloques de desarrollo y resultado principal de cada uno

Bloque	Foco	Resultado principal
0	Línea base	Diagnóstico del estado inicial y registro de deuda técnica
1	Seguridad crítica	Cierre de riesgos de dependencias y credenciales
2	Diseño de datos	Esquema relacional documentado y migraciones reproducibles
3	Calidad	Suite de pruebas e integración continua en operación
4	Persistencia y contenedores	Migración completa a PostgreSQL y entorno reproducible
5	Colaboración y observabilidad	Trabajo compartido, instrumentación y medición
6	Publicación	Despliegue con dominio propio, versión etiquetada y manifiesto

2. Control de versiones y trazabilidad

Todo el trabajo se realizó sobre un repositorio de control de versiones alojado en GitHub. Cada cambio se incorporó mediante confirmaciones descriptivas que

explican no solo qué se modificó sino por qué, criterio adoptado porque el qué resulta legible en el propio cambio mientras que el porqué se pierde si no se escribe.

3. Desarrollo guiado por pruebas

Las funcionalidades con reglas de negocio no triviales se abordaron escribiendo primero la prueba que describe el comportamiento esperado. Este orden obliga a definir el comportamiento antes de implementarlo y produce, como efecto secundario, una descripción ejecutable de las reglas del sistema.

El siguiente fragmento ilustra el criterio aplicado a los recordatorios recurrentes. La prueba no verifica la implementación interna, sino una regla de negocio observable: que la cuenta para el primer recordatorio comience en el momento de creación de la tarea y no en un valor nulo.

```
it('la cuenta arranca al crear la tarea, no en «nunca」', async () => {
  const db = getDb();
  const reciennacida = await crearTarea('Recién creada', 'urgente',
    { creadaHace: '1 minute' });
  const pendientes = await getTasksNeedingNudge(db);
  expect(pendientes.map(t => t.task_id)).not.toContain(reciennacida);
});
```

Figura 4. Prueba de la regla de conteo de recordatorios (tests/taskNudges.test.js)

Un segundo ejemplo corresponde a un defecto detectado por una prueba antes de llegar a producción. La función que determina la hora local devolvía el valor veinticuatro a medianoche, en lugar de cero, con la consecuencia de que un intervalo de silencio definido entre las cero y las siete horas no habría incluido la medianoche y el sistema habría enviado avisos a esa hora.

```
export function hourInAppTimeZone(date = new Date()) {
  const formato = new Intl.DateTimeFormat('en-US', {
    timeZone: APP_TIME_ZONE, hour: '2-digit', hourCycle: 'h23',
  });
  return Number(formato.format(date)) % 24;
}
```

Figura 5. Corrección del cálculo de la hora local (src/lib/taskNudges.js)

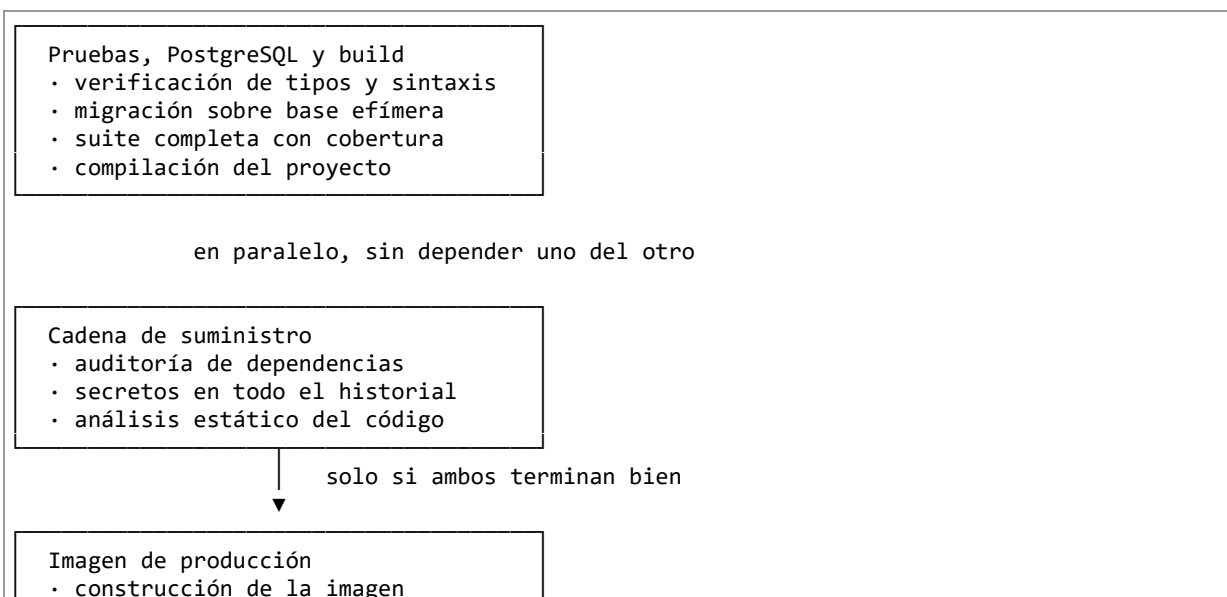
4. Estrategia de pruebas

La suite se ejecuta contra una base de datos PostgreSQL real. Para impedir que una ejecución destruya datos de desarrollo, el arranque exige que el nombre de la base de pruebas termine en un sufijo determinado y se niega a operar en caso contrario. Los servicios externos se sustituyen por dobles controlados, de modo que la ejecución sea determinista, no consuma cuota y no dependa de la red.

Las pruebas de los puntos de acceso web no levantan un servidor: importan directamente la función que atiende la ruta y le entregan un objeto de petición construido en el propio caso de prueba, técnica que reduce el tiempo de ejecución y elimina la dependencia de puertos disponibles.

5. Integración continua

El proceso automatizado se ejecuta con cada incorporación de cambios y se organiza en tres trabajos. Dos de ellos se ejecutan en paralelo y el tercero solo se inicia si ambos terminan satisfactoriamente, de modo que nunca se construye ni se publica una imagen a partir de código que no superó las verificaciones.



- migración con la propia imagen
- arranque y consulta a la sonda
- análisis de vulnerabilidades
- publicación en el registro

Figura 6. Organización del proceso de integración continua en tres trabajos

Dos decisiones de este diseño merecen explicación. La primera es que el trabajo de imagen aplica las migraciones utilizando la propia imagen construida: con ello se verifica el artefacto que efectivamente se despliega y no únicamente el código fuente. La segunda es que el paso final declara de forma explícita si el conjunto de cambios incorporado contenía migraciones de esquema, de modo que quien realice el despliegue sepa sin ambigüedad si debe generar una copia de seguridad previa.

La búsqueda de secretos se realiza sobre la totalidad del historial del repositorio y no solo sobre los archivos actuales, porque una credencial incorporada en un cambio antiguo permanece accesible aunque se haya eliminado después.



Figura 7. Ejecución del proceso de integración continua en estado satisfactorio, con sus tres trabajos

6. Instrumentación y medición

Para medir el rendimiento con criterio se instrumentó la aplicación de modo que cada solicitud emita un evento estructurado, y se desarrolló un programa de medición que ejecuta un escenario repetible y calcula percentiles y tasa de error sobre una muestra mínima de veinte peticiones.

```
{ "ts": "2026-08-14T02:52:35.328Z", "level": "info", "event": "http_request",  
  "request_id": "a361aae4-3edf-4469-94d3-bcb645de4800", "method": "GET",  
  "route": "/api/tasks", "status": 200, "outcome": "success", "duration_ms": 24,  
  "app_version": "dev", "db_queries": 2, "db_duration_ms": 18.93 }
```

Figura 8. Evento de registro emitido por solicitud

El procedimiento de medición consistió en fijar el escenario, descartar las primeras peticiones para evitar el efecto del calentamiento, ejecutar tandas con distintos niveles de concurrencia y comparar el tiempo observado por el cliente con el registrado por el servidor. Esta comparación fue la que permitió localizar el verdadero cuello de botella.

7. Despliegue

El despliegue se realiza mediante contenedores descritos en un archivo de composición versionado. La base de datos no publica puertos hacia el exterior y la aplicación escucha únicamente en la interfaz local, quedando accesible a través de un proxy inverso que termina la conexión cifrada.

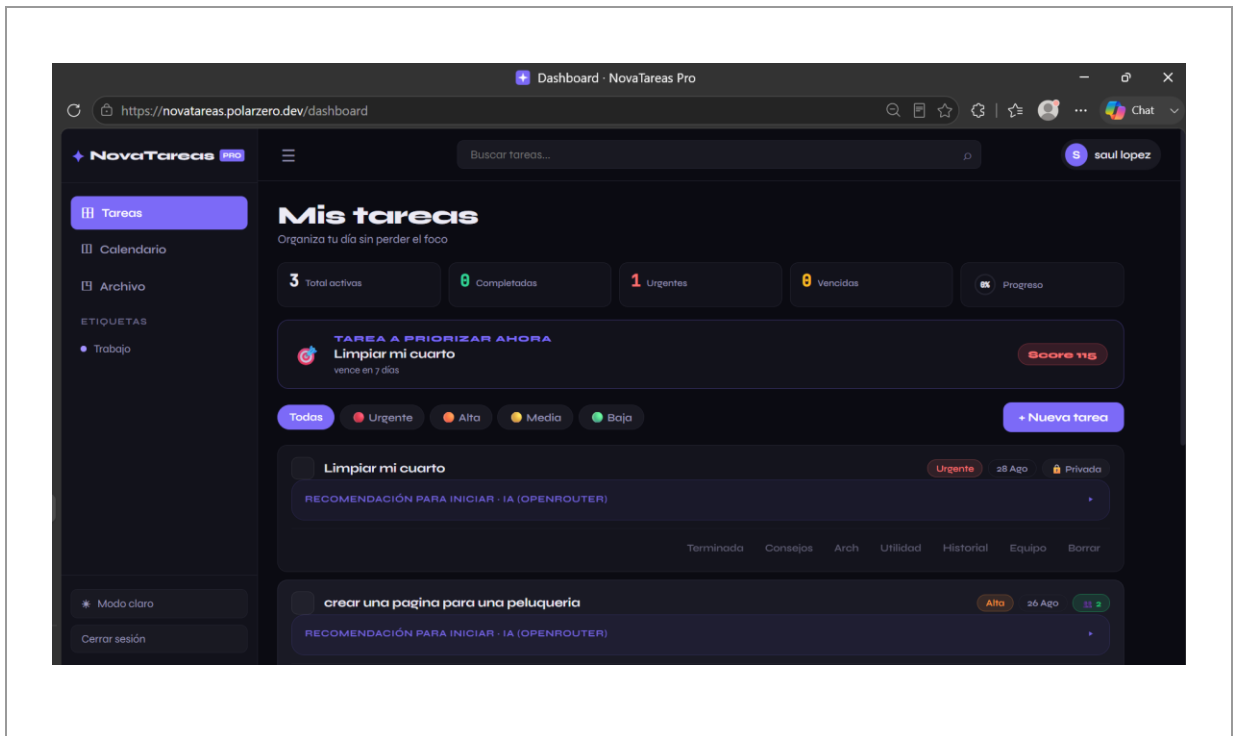


Figura 9. Aplicación en funcionamiento en <https://novatareas.polarzero.dev>

RESULTADOS ALCANZADOS

1. Producto publicado y accesible

El sistema se encuentra publicado y operativo. Su estado puede verificarse en cualquier momento mediante las siguientes peticiones, que no requieren autenticación.

```
$ curl https://novatareas.polarzero.dev/api/v1/health/ready
{"status":"ok","checks":{"database":true},"latency_ms":1}

$ curl https://novatareas.polarzero.dev/api/v1/metadata
{"service":"novatareas-ai","version":"1.0.0", ...}
```

Figura 10. Comprobación del estado del sistema publicado

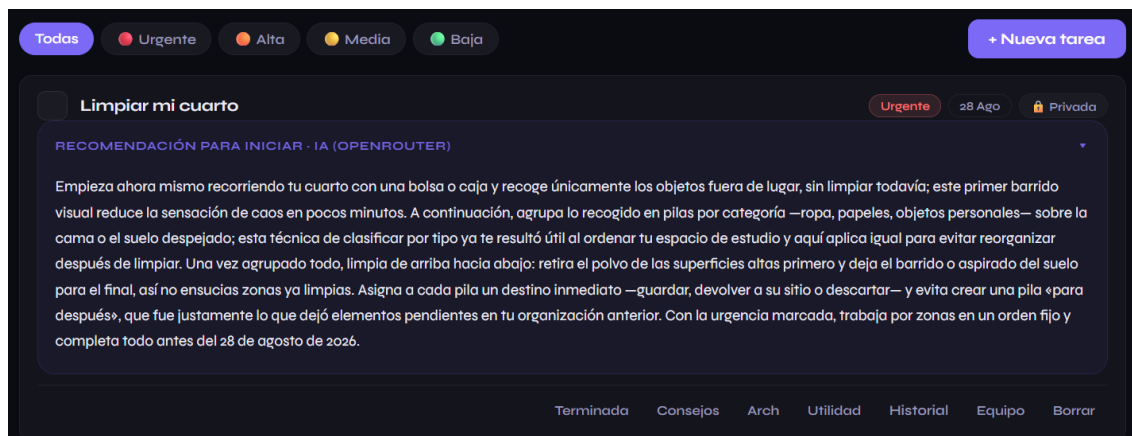


Figura 11. Panel principal con tareas, prioridades y recomendación generada

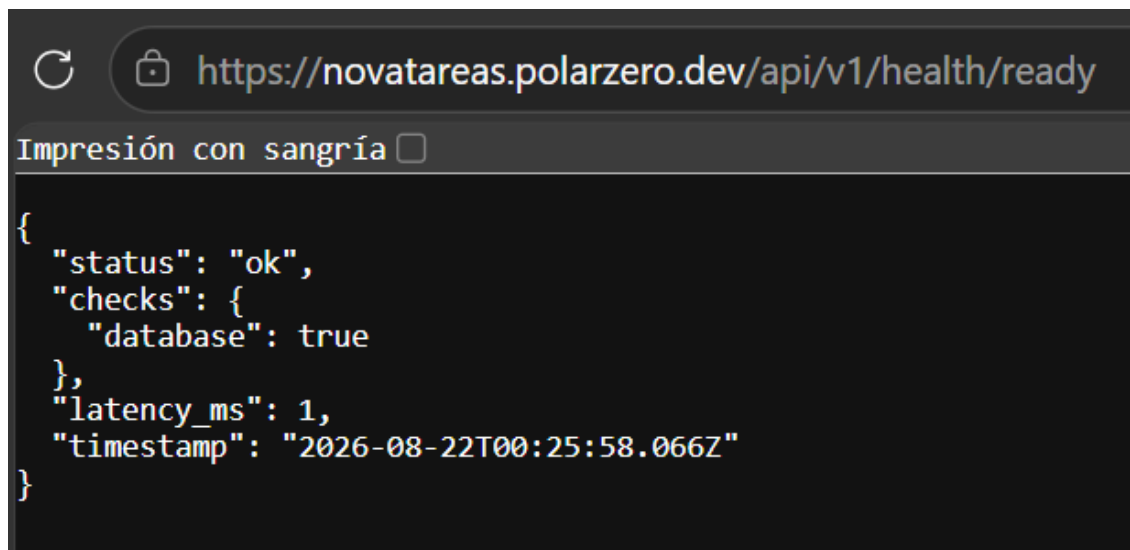


Figura 12. Respuesta de la sonda de estado en el navegador

2. Calidad verificada

Tabla 5. Indicadores de calidad del software al cierre del proyecto

Indicador	Valor
Pruebas automatizadas	33 archivos de prueba ejecutados en cada cambio
Motor sobre el que se ejecutan	PostgreSQL 16 real
Pruebas adversariales	10 casos de acceso o manipulación indebida
Servicios externos invocados durante las pruebas	Ninguno
Estado del proceso de integración continua	Satisfactorio en sus tres trabajos
Migraciones de esquema versionadas	8, idempotentes y verificadas

Las pruebas adversariales comprueban que permanezcan bloqueadas operaciones concretas: acceder a la tarea de otro usuario, editar sin el nivel de permiso adecuado, eliminar una tarea sin ser propietario, reutilizar un enlace de invitación ya consumido, deducir la existencia de una cuenta a partir de la respuesta de recuperación, exceder el límite de intentos mediante peticiones simultáneas, invocar

procesos programados sin credencial y cargar un archivo ejecutable presentado como imagen.

3. Rendimiento medido

Se estableció una línea base sobre el flujo más utilizado del sistema, el listado de tareas del panel con sesión iniciada, mediante treinta peticiones secuenciales por tanda y tres peticiones de calentamiento descartadas.

Tabla 6. Línea base de rendimiento del flujo crítico según nivel de concurrencia

Concurrencia	Mediana (ms)	Percentil 95 (ms)	Máximo (ms)	Tasa de error	Peticiones por segundo
1	16,19	23,74	49,39	0 %	56,4
4	31,36	87,60	109,83	0 %	104,4
8	73,37	208,76	227,88	0 %	93,0
16	172,05	291,81	546,09	0 %	71,8

La tasa de error fue del cero por ciento en la totalidad de las tandas.

4. Identificación del cuello de botella

La comparación entre el tiempo observado por el cliente y el registrado por el servidor permitió descomponer la latencia y localizar dónde se consumía realmente.

Tabla 7. Descomposición de la latencia mediana observada por el cliente

Tramo	Tiempo aproximado	Proporción
Fuera de la aplicación (red del entorno de desarrollo)	10,9 ms	73 %
Aplicación sin acceso a base de datos	1,0 ms	7 %
Consultas a PostgreSQL (dos por petición)	3,0 ms	20 %

El hallazgo resultó contraintuitivo: casi tres cuartas partes del tiempo percibido se consumían antes de que la petición alcanzara el proceso de la aplicación. Sin instrumentación, el esfuerzo de optimización se habría dirigido al código, que representaba apenas el siete por ciento del total.

5. Mejora aplicada y verificada

A partir del diagnóstico se aplicó una mejora concreta: servir la aplicación compilada en lugar del servidor de desarrollo, que transformaba módulos en cada petición.

Tabla 8. Comparación de rendimiento antes y después de la mejora aplicada

Métrica (conurrencia 1)	Servidor de desarrollo	Aplicación compilada	Variación
Mediana	20,67 ms	16,19 ms	-21,7 %
Percentil 95	24,56 ms	23,74 ms	-3,3 %
Peticiones por segundo	47,9	56,4	+17,7 %

El resultado se confirmó en dos ejecuciones independientes, con variaciones del veintiuno y del veintidós por ciento, lo que descarta que se trate de ruido de medición.

Se evaluó además una segunda mejora que finalmente se descartó: almacenar temporalmente la validación de sesión para eliminar una de las dos consultas por petición. El ahorro estimado era de aproximadamente un milisegundo y medio sobre quince, mientras que el costo consistía en retrasar la revocación inmediata de sesiones al cambiar la contraseña. La evidencia no justificaba asumir ese costo de seguridad.

6. Plan de escalabilidad derivado de la medición

Las mediciones bajo concurrencia mostraron que un proceso satura entre sesenta y cinco y ciento cinco peticiones por segundo, y que la latencia crece de forma

acusada a partir de cuatro peticiones simultáneas. Sobre esa base se definieron palancas de escalado asociadas a indicadores observables, de modo que la decisión de escalar responda a una señal medida y no a una impresión.

Tabla 9. Palancas de escalabilidad y señal que activa cada una

Palanca	Cuándo aplicarla	Indicador
Añadir instancias de la aplicación	Ante aumento sostenido de concurrencia	Percentil 95 superior a 300 ms con uso alto de procesador
Almacenar recomendaciones equivalentes	Cuando se repitan peticiones de IA similares	El tiempo de IA domina la duración total
Procesamiento en segundo plano	Si la generación se vuelve masiva y sincrónica	Percentil 95 del tiempo de IA superior a 5 s sostenido
Réplica de lectura de la base	Cuando la base deje de ser el 20 % del tiempo	Tiempo de base superior al 50 % de la duración

7. Evolución respecto al estado inicial

Tabla 10. Comparación entre el estado inicial del proyecto y el estado al cierre

Aspecto	Estado inicial	Estado al cierre
Base de datos	SQLite, considerada provisional	PostgreSQL 16 como motor único
Pruebas automatizadas	55 casos en 7 archivos	33 archivos de prueba ejecutados en cada cambio
Integración continua	Inexistente	Automatizada, con verificación de la imagen desplegable
Capacidad de IA	Acoplada a la interfaz	Desacoplada y expuesta como servicio
Medición de calidad de las recomendaciones	Inexistente	Valoración del usuario que realimenta la instrucción
Límites de intentos	En memoria del proceso	Persistidos en base de datos
Recuperación de cuentas	Preguntas de seguridad	Correo verificado con enlace de un solo uso
Trabajo sobre una tarea	Individual	Colaborativo con tres niveles

		de acceso
Observabilidad	Inexistente	Evento por petición y línea base medida
Disponibilidad	Sin publicar	En línea con dominio y certificado propios

8. Limitaciones del resultado alcanzado

El proyecto declara las siguientes limitaciones, cuya omisión daría una imagen inexacta de lo logrado.

La publicación de versiones conserva pasos manuales. El proceso automatizado construye y verifica la imagen desplegable, pero no la despliega. El sistema opera además sobre una única instancia, sin conmutación automática ante fallo del servidor, y el componente de mensajería no admite duplicación por la naturaleza del mecanismo de recepción que emplea.

El periodo de observación con estudiantes destinado a medir el efecto sobre hábitos de estudio y calificaciones no se completó, de modo que el impacto académico de la herramienta permanece sin cuantificar.

La integración con calendarios externos está construida y probada en sus mecanismos de autorización y cifrado de credenciales, pero no se conectó a la interfaz principal. La recuperación semántica sobre el historial funciona, aunque no todas las rutas del sistema la aprovechan todavía. La cobertura de pruebas se mide pero no se exige mediante un umbral que interrumpa el proceso automatizado. Y la pantalla principal quedó concentrada en un archivo de dimensiones excesivas cuya división facilitaría el mantenimiento.

CONCLUSIONES GENERALES

1. Aporte a la comunidad

El proyecto pone a disposición de los estudiantes de la Universidad Gerardo Barrios una herramienta gratuita, en español y accesible desde internet, que atiende una necesidad verificada con evidencia propia dentro de esa misma comunidad. A diferencia de las alternativas comerciales, no exige un pago recurrente en moneda extranjera ni presupone dominio del inglés, dos barreras concretas para la población objetivo.

El aporte no se limita a la aplicación. El repositorio queda públicamente disponible con su documentación técnica, su conjunto de pruebas y su proceso de integración continua, de modo que puede servir como material de referencia para estudiantes de la institución que aborden proyectos con requisitos similares.

2. Resultados concretos obtenidos

El sistema está publicado y operativo, con versión declarada y consultable, y responde satisfactoriamente a sus sondas de estado. Cuenta con una suite de pruebas automatizadas ejecutada contra el motor real en cada cambio del código, y con un proceso automatizado que verifica no solo el código sino el artefacto que efectivamente se despliega.

Se estableció una línea base de rendimiento con percentiles y tasa de error, a partir de la cual se identificó el cuello de botella real del flujo crítico y se aplicó una mejora que redujo la latencia mediana en aproximadamente un veintiuno por ciento, confirmada en mediciones independientes.

Se construyó un mecanismo de degradación progresiva que mantiene el servicio disponible ante la caída del proveedor de inteligencia artificial, y se documentó un procedimiento de vuelta atrás con su limitación explícita respecto de las migraciones de esquema.

3. Conocimientos adquiridos

El desarrollo dejó tres aprendizajes que el equipo considera transferibles a proyectos futuros.

El primero es que la medición precede a la optimización. La intuición del equipo situaba el cuello de botella en el código de la aplicación; la instrumentación demostró que casi tres cuartas partes del tiempo se consumían fuera de él. Sin esa evidencia, el esfuerzo se habría invertido en el lugar equivocado y el resultado habría sido nulo.

El segundo es que una dependencia externa debe tratarse como un modo de fallo previsible y no como una condición garantizada. Diseñar la degradación progresiva desde el inicio convirtió lo que habría sido una interrupción del servicio en una reducción temporal de calidad, transparente para el usuario y trazable para el equipo.

El tercero es que los controles de seguridad requieren pruebas que los vigilen. Un control funciona en silencio: no produce ninguna manifestación observable mientras cumple su función, de modo que puede desaparecer durante una modificación sin que nadie lo advierta hasta que alguien lo explote. Las pruebas adversariales convierten esa ausencia silenciosa en un fallo visible e inmediato.

4. Trabajo futuro

Las líneas de continuación identificadas, en orden de prioridad, son la automatización del último tramo del despliegue; la realización del periodo de observación con estudiantes para cuantificar el efecto académico; la conexión de la integración de calendarios a la interfaz principal; la agregación de las valoraciones recibidas en un informe que muestre la evolución de la calidad de las recomendaciones; la incorporación de la recuperación semántica en todas las rutas que pueden beneficiarse de ella; y la división de la pantalla principal en módulos de menor tamaño.

REFERENCIAS ESTUDIADAS

Bibliografía

Allen, D. (2015). *Getting Things Done: The Art of Stress-Free Productivity* (edición revisada). Penguin Books.

Beck, K. (2002). *Test-Driven Development: By Example*. Addison-Wesley.

Beyer, B., Jones, C., Petoff, J. y Murphy, N. R. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media.

Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks. *Communications of the ACM*, 13(6), 377–387.

Humble, J. y Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33.

Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2.^a edición). O'Reilly Media.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N. et al. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30.

Referencias web

Astro. *Documentación oficial del framework*. <https://docs.astro.build>

Docker. *Documentación oficial*. <https://docs.docker.com>

Fowler, M. (2015). *MonolithFirst*. <https://martinfowler.com/bliki/MonolithFirst.html>

GitHub. *Documentación de GitHub Actions*. <https://docs.github.com/actions>

Keep a Changelog. *Versión 1.1.0*. <https://keepachangelog.com/es-ES/1.1.0/>

OWASP Foundation. *OWASP Top Ten*. <https://owasp.org/www-project-top-ten/>

OWASP Foundation. *Cross-Site Request Forgery Prevention Cheat Sheet*.
<https://cheatsheetseries.owasp.org>

PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*.
<https://www.postgresql.org/docs/16/>

Preston-Werner, T. *Semantic Versioning 2.0.0*. <https://semver.org/lang/es/>

Vitest. *Documentación oficial*. <https://vitest.dev>

Z.ai. *Documentación de la interfaz de programación de los modelos GLM*.
<https://docs.z.ai>

Fuentes propias

Equipo 3 (2026). *Encuesta sobre organización de tareas y carga académica aplicada a estudiantes de la Universidad Gerardo Barrios*. Febrero de 2026.

Equipo 3 (2026). *Repositorio del proyecto NovaTareas Pro*.
<https://github.com/Saul1hdz/NovaTareas>